LeetCode Daily Question — October 16, 2026

2916. Subarrays Distinct Element Sum of Squares II

Difficulty: Hard

Problem

You are given a 0-indexed integer array nums.

The distinct count of a subarray of nums is defined as:
Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j].

Return the sum of the squares of distinct counts of all subarrays of nums.

Since the answer may be very large, return it modulo 10^9 + 7.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:
Input: nums = [1,2,1]
Output: 15
Explanation: Six possible subarrays are:
[1]: 1 distinct value
[2]: 1 distinct value
[1]: 1 distinct value
[1,2]: 2 distinct values
[2,1]: 2 distinct values
[1,2,1]: 2 distinct values
The sum of the squares of distinct counts in all subarrays is 15.

Example 2:
Input: nums = [2,2]
Output: 3

Constraints:
1 <= nums.length <= 10^5
1 <= nums[i] <= 10^5

Approach

Maintain, for every left endpoint, the distinct count of the subarray ending at the current right endpoint. When nums[right] repeats a value last seen at index p, only subarrays starting after p gain one distinct value. Use a lazy segment tree to add 1 over that range. For a count changing from c to c + 1, its square increases by 2c + 1. After each update, add the global sum of squares to the answer.

Java Solution

```
class Solution {
    private static final int MOD = 1_000_000_007;

    private long[] count;
    private long[] squareSum;
    private long[] lazyAdd;

    public int sumCounts(int[] nums) {
        int n = nums.length;
        count = new long[4 * n];
        squareSum = new long[4 * n];
        lazyAdd = new long[4 * n];

        int[] last = new int[100_001];
        java.util.Arrays.fill(last, -1);

        long answer = 0;

        for (int right = 0; right < n; right++) {
            int value = nums[right];
            int start = last[value] + 1;

            update(1, 0, n - 1, start, right, 1);
            answer = (answer + squareSum[1]) % MOD;

            last[value] = right;
        }

        return (int) answer;
    }

    private void update(
            int node,
            int nodeLeft,
            int nodeRight,
            int updateLeft,
            int updateRight,
            int delta) {

        if (updateRight < nodeLeft || nodeRight < updateLeft) {
            return;
        }

        if (updateLeft <= nodeLeft && nodeRight <= updateRight) {
            apply(node, nodeLeft, nodeRight, delta);
            return;
        }

        pushDown(node, nodeLeft, nodeRight);

        int middle = nodeLeft + (nodeRight - nodeLeft) / 2;

        update(2 * node, nodeLeft, middle,
                updateLeft, updateRight, delta);
        update(2 * node + 1, middle + 1, nodeRight,
                updateLeft, updateRight, delta);

        pullUp(node);
    }

    private void apply(
            int node,
            int left,
            int right,
            long delta) {

        long length = right - left + 1;

        squareSum[node] = (squareSum[node]
                + 2 * delta % MOD * count[node] % MOD
                + delta * delta % MOD * length % MOD) % MOD;

        count[node] = (count[node] + delta * length) % MOD;
        lazyAdd[node] = (lazyAdd[node] + delta) % MOD;
```

```
    }

    private void pushDown(int node, int left, int right) {
        if (lazyAdd[node] == 0) {
            return;
        }

        int middle = left + (right - left) / 2;

        apply(2 * node, left, middle, lazyAdd[node]);
        apply(2 * node + 1, middle + 1, right, lazyAdd[node]);

        lazyAdd[node] = 0;
    }

    private void pullUp(int node) {
        count[node] = (count[2 * node] + count[2 * node + 1]) % MOD;
        squareSum[node] = (squareSum[2 * node]
                + squareSum[2 * node + 1]) % MOD;
    }
}
```